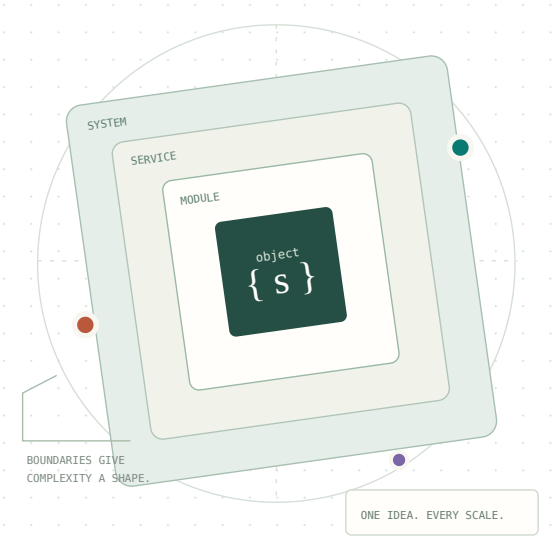
THE SYSTEM DESIGN ATLAS

From objects to *planet-scale systems.*

A system is more than its boxes and arrows. Learn the promises, mechanisms, and trade-offs that connect a single function to a distributed world.

Build the simple version. Find its limit. Understand the next design.



A CONCEPTUAL COVER STUDY · AUTHOR-CREATED

22	04	16	12
UNITS MAPPED	CHAPTERS TO READ	ORIGINAL FIGURES	CASE STUDIES PLANNED

- **A book built in stages.** The complete 99-chapter blueprint is ready. Unit 1 is written and illustrated; Units 2–22 remain planned. See exactly what is complete → · Download the foundations PDF ↓

THE TERRITORY

Six parts. One connected story.

See the concept map ↗

PART 01

System design foundations

Learn to see the system before scaling it.

4 chapters ready ↗

PART 02

Low-level design & software architecture

Organize behavior, dependencies and change.

Blueprint ↗

PART 03

Data, storage & scalability

Make access patterns and capacity visible.

Blueprint ↗

PART 04

Distributed systems & reliability

Reason about uncertainty and partial failure.

Blueprint ↗

PART 05

Security & production engineering

Operate, protect and validate the design.

Blueprint ↗

PART 06

Complete system design case studies

Twelve systems, examined from end to end.

12 case studies planned ↗

READY TO READ / UNIT 01

Before you scale a system, learn to *see one*.

Follow a to-do application from a working script to an explicit service contract. Lose a write, isolate a responsibility, record a decision, and trace a request.

Open the foundations unit ↗

1.1 A system is a promise ↗

1.2 Boundaries that make change local ↗

1.3 Requirements become decisions ↗

1.4 One application, many views ↗

---

01 / THE PROBLEM

Begin where it breaks.

A concept earns its place by solving an observable problem.

02 / THE MECHANISM

Make execution visible.

Inspect the state, ownership, messages, and ordering.

03 / THE JUDGMENT

Keep the trade-off.

Understand what improves, what it costs, and when to choose differently.

A system is a promise

Prerequisite: Basic programming 4 original figures · 1 workbench

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / infrastructure

Gray: external / inactive Solid arrow: action / data · dashed: dependency or zoom · dotted: acknowledgment

You write a small program that remembers tasks. It accepts “Read the design notes,” displays a success message, and lists the task. Then you close the program. The next morning, the list is empty.

The program did exactly what its instructions said. The system failed the promise the user thought it made. System design begins in that gap: between executing code and delivering dependable behavior in a real environment.

> THE QUESTION
>
> What must remain true after the happy path ends?

1.1.1 · Start with behavior, then draw the boundary

A **software system** is a collection of cooperating elements, considered within a boundary, that delivers behavior to its environment. The elements can include code, processes, stored data, devices, and external services. A useful boundary is chosen for a question: the application process is one system for a runtime discussion; the application plus database is another for a user-facing reliability discussion. Neither boundary is universally correct.

Our first scope is deliberately small. One person can create a task, list tasks, and mark a task complete. A successful creation must survive an ordinary application restart. We assume one machine, a functioning local disk, and one user. We have not promised survival of disk destruction, remote sharing, or global availability. Those are different requirements with different costs.

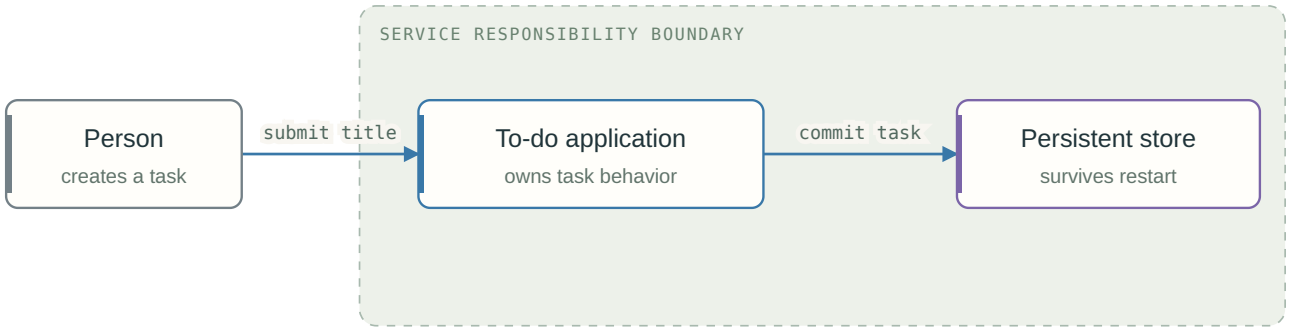


FIG. 1.1.1 **The boundary follows the promise**

A user asks the application to remember a task. The store is inside the service responsibility boundary even if a different team operates it.

Why this view: It separates the user-visible obligation from the process that happens to run the code. Author-created illustration.

Designing this system means choosing responsibility, representation, interactions, and operating assumptions so that these behaviors hold. A diagram is evidence about a design, not the design itself. A box labeled “database” does not answer when success is returned, what happens on a failed write, or who restores a broken file. Those choices are part of the system.

1.1.2 · The simplest working implementation

A collection in memory is sufficient if tasks need exist only during one run. This miniature uses explicit identifiers so the state changes are easy to inspect. The caller chooses an unused positive integer; a production API would need an identity policy. The function returns a fresh snapshot so callers cannot silently change internal state through the returned object.

```
tasks = {}

def create(task_id, title):
    title = title.strip()
    if not title:
        raise ValueError("title must not be empty")
    if task_id in tasks:
        raise ValueError("task id already exists")
    tasks[task_id] = {"title": title, "done": False}
    return {"id": task_id, **tasks[task_id]}

print(create(7, "  Read  "))
# {'id': 7, 'title': 'Read', 'done': False}
```

Completion is the transition `tasks[7]["done"] = True`, after checking that task 7 exists. Repeating that assignment has the same task-state effect as doing it once. Repeating an unkeyed creation is different: it may create another task. The downloadable example implements both creation and completion with a commit before returning.

The mechanism is straightforward: normalize input, check the invariant, insert a record, and return the result. An **invariant** is a condition required to hold in every valid state: here, task identifiers are unique and titles are nonempty after trimming. This implementation assumes sequential calls. Under concurrent execution, checking an identifier and then inserting it is a compound operation that would need coordination. We will not claim it is thread-safe.

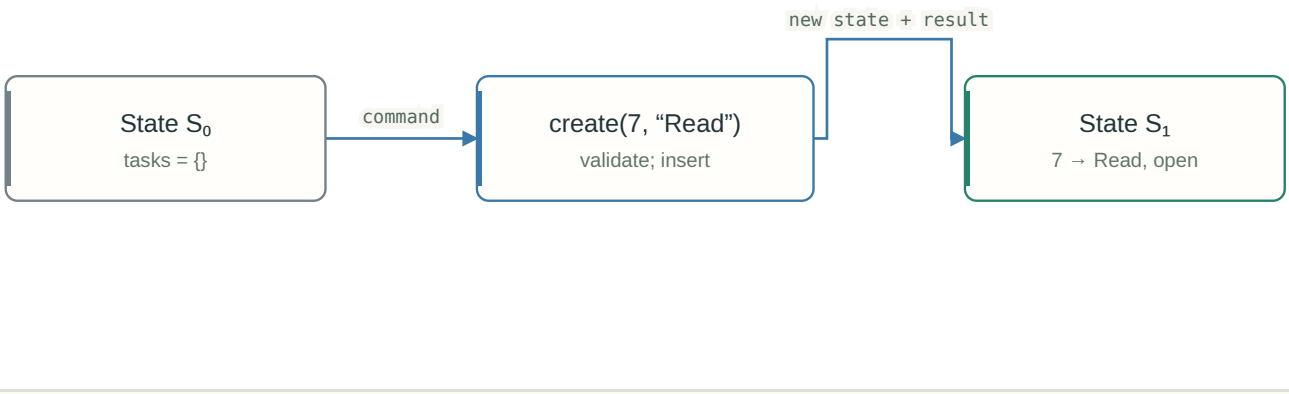


FIG. 1.1.2 **Data becomes state through a transition**

Given an empty collection and the create command, the state gains task 7 and the caller receives its identifier. Repeating that command is a separate design question.

Why this view: A concrete transition exposes both the state owner and the effect that a retry could duplicate. Author-created illustration.

> A SMALL FORMAL MODEL
>
> Let `S` be the map from task identifiers to records, and `c` a command. Model execution as `T(S, c) → (S′, result)`. Validation failure leaves the state unchanged. A create with a new identifier extends the map by exactly one entry. This is a useful model of behavior; it abstracts away scheduling, I/O, and crashes.

Data is a representation: the characters "Read," an identifier, a Boolean. **State** is the information whose current value affects future behavior: the set of tasks currently stored. **Computation** transforms or examines data, such as validating a title. **Communication** transfers information across a boundary, such as a function call, a filesystem operation, or a network request. A single operation can involve all four. Distinguishing them helps locate both ownership and failure.

1.1.3 · Components, interfaces, and contracts

A **component** is a unit of responsibility chosen for the design. In the next chapter we will separate input handling, task rules, and storage. A component does not necessarily mean an independently deployed service. It might be a module, an object, or a group of functions in one executable.

An **interface** names the operations a collaborator can use. A **contract** adds meaning: accepted inputs, observable outputs, failure outcomes, and obligations. The signature `create(title) → Task` is insufficient. Does it trim whitespace? Is an empty title rejected? Does returning imply persistence? What does a timeout mean? A caller must know these answers to use the operation correctly.

Table 1.1.1 · A small creation contract

Clause	Chosen behavior	Evidence
Input	Trimmed title must contain 1–200 characters.	Whitespace-only and oversized titles are rejected.
Success	A new task is committed before returning its identifier.	Restart application and read back the identifier.
Storage failure	No success response; caller gets a storage error.	Inject a write failure and inspect the result.
Lost response	Outcome may be unknown to caller.	Commit, then interrupt the response.
Retry	A fresh create may duplicate intent.	Repeat the request; inspect both rows.

Notice that an error at the communication boundary need not mean the business action failed. A client can time out after a task was saved. To guarantee a deduplicated retry, we would need an additional identity and retention contract. Merely labeling a function “idempotent” does not implement that mechanism. Chapter 8.2 is assigned that deeper treatment.

1.1.4 · High-level and low-level views meet at the contract

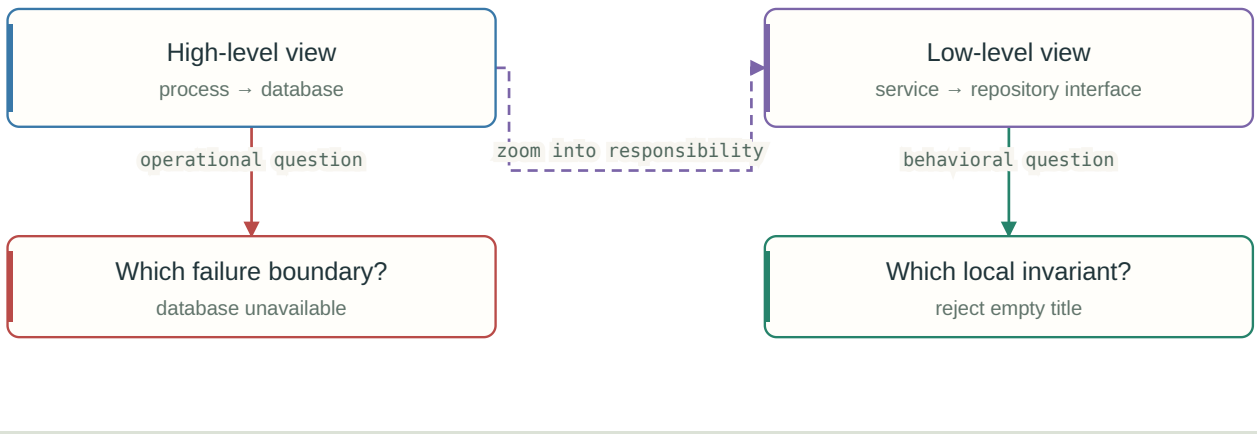


FIG. 1.1.3 **HLD and LLD ask connected questions**

The high-level view places persistent state and communication boundaries. The low-level view explains how application components enforce behavior inside a process.

Why this view: Both views must agree about who commits state and who reports success. Author-created illustration.

High-Level Design (HLD) commonly describes major components, data placement, communication, deployment, and quality trade-offs. **Low-Level Design (LLD)** commonly describes the internal modules, interfaces, types, algorithms, and collaboration that realize those responsibilities. **Software architecture** concerns consequential structural and behavioral choices and their rationale. Communities draw these boundaries differently. None of these labels is a universal formal partition of engineering work.

For example, HLD might say that the application commits tasks to a local store before acknowledging creation. LLD decides how the service receives a repository, validates a task, calls a transaction, and maps an error. They describe the same promise at different resolutions. A beautifully organized service class cannot correct an architecture that acknowledges volatile state as durable.

1.1.5 · Break the happy path on purpose

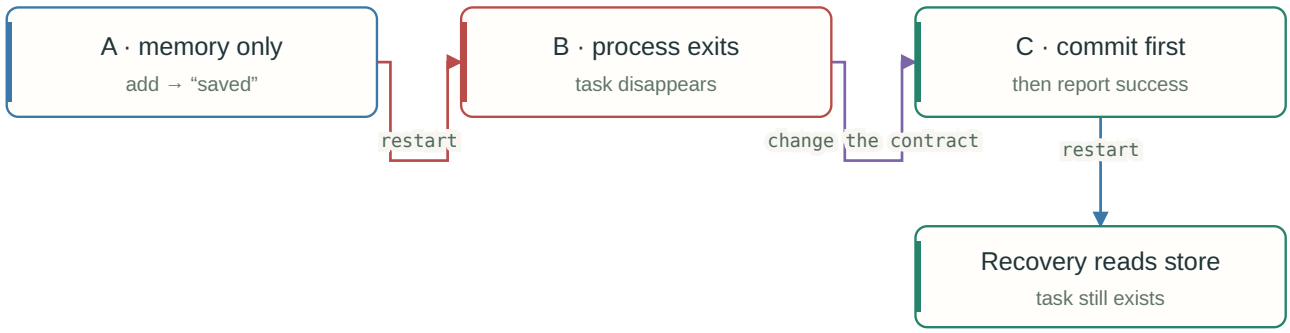


FIG. 1.1.4 **From a working demo to a recoverable write**

The naive program works within one run. A restart disproves its persistence promise. Committing before success fixes that failure under the storage assumptions; it does not by itself solve lost replies.

Why this view: The improvement is tied to a reproducible failure rather than an arbitrary desire for more components. Author-created illustration.

Restart the in-memory program and its map is initialized again. Moving the map into another class changes code organization but does not change that outcome. Writing JSON directly to a file helps with ordinary restarts, but introduces new questions: can a crash leave half a document, do concurrent writers overwrite each other, and what does “write completed” mean for the device? A durable embedded database is a reasonable next choice for this example because it provides a defined transaction mechanism. Its guarantees still depend on configuration and the storage stack. SQLite’s atomic-commit documentation explains those assumptions; it is not a guarantee against every possible hardware failure.

THE WORKBENCH · AUTHOR-CREATED MODEL

When is “saved” actually safe?

Choose the acknowledgment policy, then step through one request. Restart at any point and compare the promise with the recovered state.

Policy: C · commit before acknowledgment · 0 events executed

The caller has not received success.

APPLICATION MEMORY	COMMITTED STORE	CALLER’S KNOWLEDGE
Empty	Empty	No success received

Step the request, or restart to test the current state.

Assumptions: one request, atomic commit, intact storage. A teaching state machine; no real files are written.

The simulator deliberately models only process loss: volatile memory disappears, committed storage remains, and a commit is represented as an atomic transition. It does not simulate a filesystem, power loss, replication, or a real database engine. Use it to reason about acknowledgment ordering, then run the executable example to inspect actual commit behavior.

▼ Go deeper · Safety, progress, and observable evidence

“Never acknowledge an uncommitted task” is a safety property: it excludes a bad execution. “Eventually finish a valid request” is a progress requirement that also depends on resource availability and the environment. A program that never replies can satisfy the first property while being useless. Conversely, immediately replying to every request can provide apparent progress while violating persistence.

A system review should ask which observations would falsify each claim. Restarting the application tests one failure class. It does not test loss of the database disk. Restoring a backup tests another behavior. It does not prove that every recent write reached that backup. This habit of pairing a claim with its boundary and evidence will matter again in reliability and validation.

MISCONCEPTION

"It worked once" establishes one successful execution. It does not establish the contract for retries, restarts, bad inputs, or competing callers. Design the allowed histories, not just one screenshot of success.

FIELD NOTE

A database transaction and an HTTP response are two different events. There is no magic line of application code that makes the user's knowledge update atomically with a commit on another machine. This small gap drives much of the later discussion of idempotency and reconciliation.

Carry forward: A system is a promise made under assumptions. Find the state, name its owner, and identify the event after which success is justified.

Before continuing: If a task survives a process restart, which failure has been tested—and which three failures are still untested?

Sources & further reading

Read from introductory guidance to the primary or implementation detail relevant to this chapter. Links checked 21 October 2026.

1. Atomic Commit in SQLite ↗

   Official documentation · accessed 21 Oct 2026. Commit and operating-system assumptions.

   `Link checked · 2026-10-21`

2. RFC 9110: HTTP Semantics, June 2022 ↗

   Original standard. HTTP semantics; application commit policy is separate.

   `Link checked · 2026-10-21`

Boundaries that make change local

Prerequisite: Chapter 1.1 4 original figures · 1 workbench

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / infrastructure

Gray: external / inactive Solid arrow: action / data · dashed: dependency or zoom · dotted: acknowledgment

The to-do program now persists its list. A teammate adds a CSV export. Another adds a web form. Each copies the same rule for trimming titles, and both reach directly into the stored records. A tiny schema change breaks three features.

The immediate problem is change propagation. The next design step is to place knowledge where it can change safely.

1.2.1 · First diagnose what is coupled

Imagine one handler that parses an HTTP body, validates a title, constructs an SQL statement, commits a connection, and generates HTML. Initially this is easy to follow. With two entry points, the validation rule is copied. With CSV export, persistence layout escapes into another feature. A title-length policy is now enforced inconsistently, and changing the column name requires coordinated edits.



FIG. 1.2.1 **An innocent feature crosses every responsibility**

In the tangled implementation, formatting, business rules, and persistence all know the mutable representation. Changing a field or invariant can break several paths.

Why this view: Dependency arrows identify the concrete knowledge that makes a change spread. Author-created illustration.

Coupling is the dependency between elements: what one must know or assume about another. The number of arrows is only a clue. One dependency on a stable, narrow interface can be easier to maintain than one dependency on a mutable global structure with undocumented ordering rules. Temporal coupling—"call initialize before save"—can be dangerous even when there are few imports.

Cohesion concerns how strongly the elements within a module serve a focused purpose. A task service that validates and coordinates task operations has a coherent responsibility. A helper module containing CSV formatting, database connection management, and password logic has little such focus. These are design judgments supported by change scenarios, not universal numerical scores. High cohesion does not mean a class must contain exactly one method.

1.2.2 · Separate what changes for different reasons

Our first revision uses three responsibilities. The **input adapter** understands transport syntax. The **task service** understands task rules. The **repository** understands persistent representation. The service returns application data; adapters choose whether to render HTML, JSON, or text. This is **separation of concerns**: organizing around distinct responsibilities while making their coordination explicit.

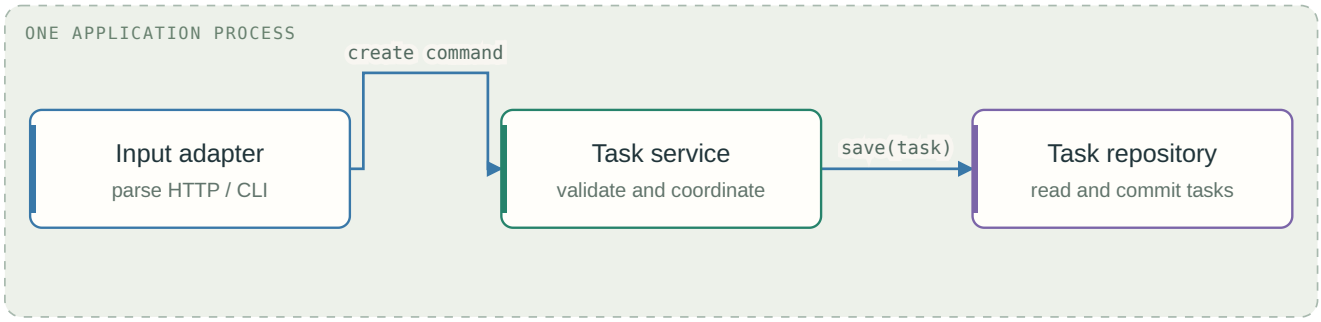


FIG. 1.2.2 **Separate responsibilities inside one process**

The adapter translates input, the service owns task rules, and the repository owns persistence operations. All three can be ordinary functions in one deployment.

Why this view: A module boundary can reduce change propagation without adding a network boundary. Author-created illustration.

```
# Language-neutral pseudocode; all calls remain in one process.
interface TaskRepository:
    insert(title) -> committed Task or StorageError
    list() -> list of Task snapshots
    set_done(id) -> committed Task or NotFound or StorageError

class TaskService:
    constructor(repository):
        self.repository = repository

    create(raw_title):
        title = trim(raw_title)
        require 1 <= length(title) <= 200
        return self.repository.insert(title)

handle_http(request, service):
    title = parse_json(request.body)["title"]
    task = service.create(title)
    return response(201, json(task))
```

The implementation must handle missing fields, invalid JSON, unsupported content types, and storage errors at the adapter boundary. We omit those mechanics in this sketch to isolate responsibility; the executable Python example provides actual validation and commit behavior without pretending to be a complete web server.

Now a command-line adapter can call the same service. A CSV exporter reads snapshots from the repository and formats them without mutating task state. The rule has one authoritative owner. If storage also enforces a constraint, that is deliberate defense of an invariant at another boundary; it is not a second, unrelated business-policy implementation.

1.2.3 · An abstraction is a promise with hidden details

Abstraction presents the behavior a caller needs while suppressing details irrelevant to that caller. The service needs “insert and return a committed task,” not “open this file, acquire this lock, and encode these bytes.” **Encapsulation** restricts access to the representation so that callers cannot bypass its invariants. An interface without encapsulation is weak: if every module can mutate the repository’s internal map, its save contract is easily circumvented.

Modularity means the system has separable units with explicit responsibilities and controlled dependencies. Files are a tool for modularity; ten files that share every internal variable are not ten independent modules in a useful engineering sense. Our test is concrete: if the persistent layout changes, which application responsibilities must change with it?

> A USEFUL ANALOGY
>
> A module interface resembles a service counter: a caller requests a result without entering the workshop. The analogy stops at failures and timing. Software callers still need to know whether an operation can block, throw, time out, or partially complete.

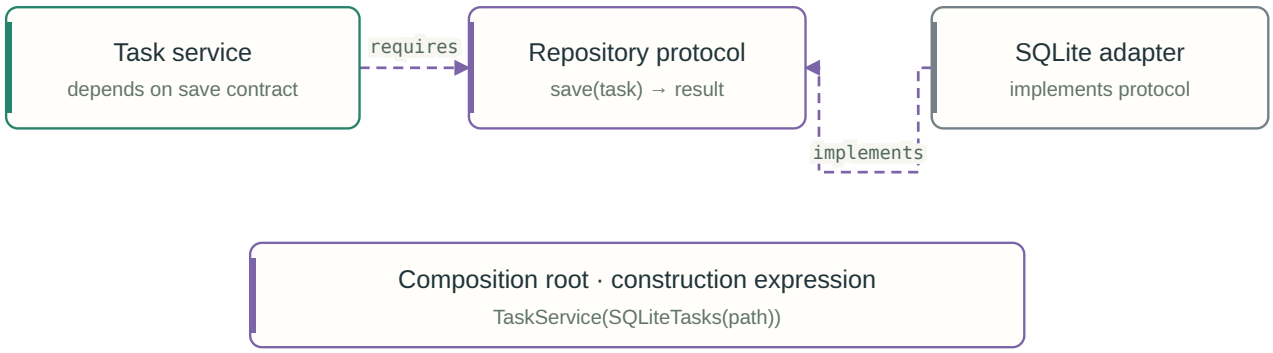


FIG. 1.2.3 **Dependency direction differs from execution direction**

Dashed arrows mean source-level dependencies. At runtime the task service calls the concrete adapter through the agreed contract; there is no extra repository server.

Why this view: It prevents the common mistake of interpreting every diagram arrow as a network request. Author-created illustration.

The diagram distinguishes source dependencies from runtime calls. The service is written against the repository contract. A concrete SQLite adapter implements it. The **composition root** is the place where the application constructs and connects these objects. At runtime, calling the interface executes the adapter. This does not require a dependency-injection container or a remote repository service. Passing an argument is sufficient.

This boundary is useful if two implementations serve a real need: an in-memory test double for fast rule tests and a persistent adapter for the application. It costs extra types, contracts, and integration testing. If the program is a 20-line disposable script, an explicit parameter and a few functions may be clearer. Do not introduce a framework just to demonstrate a principle.

1.2.4 · Follow a change through the design

Table 1.2.1 · Change propagation as design evidence

Change	Tangled implementation	Separated implementation	Remaining obligation
JSON becomes a web form.	May disturb validation and SQL.	Input adapter changes.	Adapter tests check parsing and error mapping.
Title limit becomes 200 characters.	Find all duplicated checks.	Task rule changes.	Reconcile any storage constraint and old records.

Change	Tangled implementation	Separated implementation	Remaining obligation
Database column is renamed.	Handlers and exporters know it.	Storage adapter changes.	Migration and contract tests still matter.
Create must deduplicate retries.	Touches input and stored identity.	Contract, service, and repository change.	This is legitimately a cross-boundary behavior change.

The last row matters. Good boundaries do not make all future changes local. They make dependencies honest. Adding idempotency changes what creation means, so several collaborators must agree about the key, retention, and result. Trying to hide that change inside one layer would create an inconsistent contract.

THE WORKBENCH · AUTHOR-CREATED MODEL

Follow the knowledge that must change

Scenario: Replace JSON input with a form

Tangled design: Handler · Task logic · Storage access embedded in handler

Separated design: Input adapter

Change the transport parser. The application command and persistent representation remain the same.

This is a dependency model of the worked example, not a prediction of engineering hours.

1.2.5 · Layers reduce detail, not responsibility

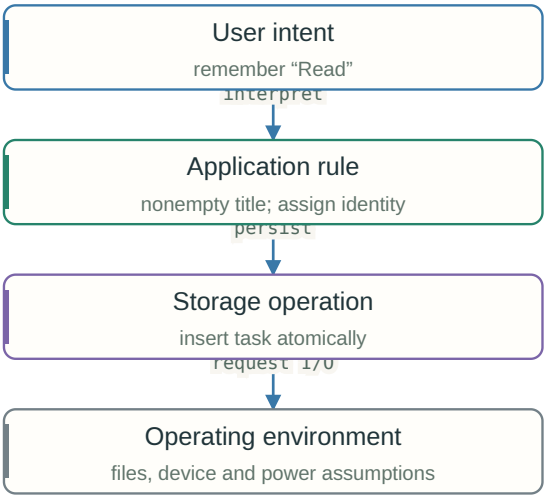


FIG. 1.2.4 **Abstraction narrows the question at each layer**

Each layer hides details but relies on a narrower promise below it. A successful application call cannot offer stronger persistence than its storage and operating assumptions support.

Why this view: Abstraction reduces what a caller must know; it cannot remove physical dependencies. Author-created illustration.

An **abstraction layer** offers one vocabulary to the layer above and relies on another below. “Remember a task” becomes an application operation, then a storage transaction, then filesystem and device actions. Different diagrams may show different layers. A source-level dependency rule need not follow the same direction as a runtime data path.

Abstractions leak when their hidden assumptions become relevant. A repository interface returning a lazy cursor may keep a database connection alive while an HTTP client slowly reads results. The service has acquired a resource-lifetime obligation it did not anticipate. A clearer contract might return a bounded materialized page or explicitly expose a streaming lifetime with cancellation. A simple method name does not make the underlying resource cost disappear.

▼ Go deeper · Behavioral substitution beats matching signatures

Suppose a memory repository and a disk repository both implement `insert(title)`. The first returns after mutating memory; the second returns after commit. They match syntactically but differ on persistence. A memory double is suitable for testing title validation, but cannot validate durability. Integration tests must exercise the actual adapter and its storage configuration.

Likewise, an adapter that accepts a transaction but commits every operation internally cannot substitute for one that promises atomic multi-operation work. The contract must identify the transaction owner. Interface compatibility includes behavior, errors, lifetime, and concurrency assumptions—not just argument types. Unit 6 will develop dependency lifetimes, and Unit 10 will develop transaction boundaries.

1.2.6 · The engineering trade-off

The revised design reduces the knowledge each collaborator needs. It adds indirection and requires someone to maintain the contracts. Prefer a boundary where there is a clear reason for independent change, an invariant to protect, or a resource whose ownership needs to be explicit. Do not maximize the number of interfaces. A module that merely forwards every call without hiding knowledge may add navigation cost without improving the design.

Operationally, these modules still share a process and a failure domain. A repository that blocks forever can stall the service. An unbounded list query can exhaust memory. Module organization helps reasoning and testing; it does not automatically deliver overload control or availability. Extracting every module into a service would add network uncertainty, deployment coordination, and observability needs. That choice requires separate evidence.

MISCONCEPTION

"Low coupling" does not mean no dependencies. A functioning system requires collaboration. Prefer dependencies whose meaning is narrow, stable enough for the task, and explicit about behavior.

RESEARCH NOTE

Parnas's 1972 modularization paper compares different ways to divide the same system. Its lasting lesson is to examine the criteria behind the division, rather than assume that one module per processing step is necessarily best. Our to-do example is an original application of that design question. Read the original paper record ↗

Carry forward: Put knowledge behind a contract where there is a reason for it to change independently. Then test both the local rule and the real integration.

Before continuing: If a repository starts returning results before they commit, which callers can break even though its method signature stays the same?

Sources & further reading

Read from introductory guidance to the primary or implementation detail relevant to this chapter. Links checked 21 October 2026.

1. On the criteria to be used in decomposing systems into modules — D. L. Parnas, 1972 ↗

   Original research · ACM DOI. Module decomposition criteria; metadata and abstract verified.

   `Link checked · 2026-10-21`

2. Inversion of Control Containers and the Dependency Injection pattern — Martin Fowler, 2004 ↗

   Original engineering article. Dependency injection and wiring.

   `Link checked · 2026-10-21`

Requirements become decisions

Prerequisite: Chapter 1.2 4 original figures · 1 workbench

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / infrastructure

Gray: external / inactive Solid arrow: action / data · dashed: dependency or zoom · dotted: acknowledgment

A request to “make the app production-ready” is not yet a design input. Should it survive a process crash, a destroyed laptop, a failed region, or a malicious administrator? Should a task appear in ten milliseconds or two seconds?

Each interpretation produces a different system. Requirements turn an argument about preferred technologies into a discussion about observable outcomes.

1.3.1 · Separate behavior from the conditions around it

A **functional requirement** specifies behavior the system must provide: create a task, list tasks, and complete a task. A **nonfunctional requirement** constrains the quality or conditions of that behavior: response latency, persistence, access isolation, operating cost, or ease of change. The distinction is useful but imperfect. Authorization may be specified as a function and also evaluated as a security property. Focus on the contract, not on winning a classification debate.

For the shared version of our application, let us write a concrete brief. These are **illustrative assumptions**, not measured facts: a team of 20 people, at most 5 requests per second during normal use, a single application machine, and a small operational budget. Users can create, list, and complete their own tasks. The service rejects malformed input. A reported successful creation survives restarting the application process while the storage system remains intact. List requests should complete within 300 ms for at least 95% of requests under the stated test workload.

That last requirement needs a measurement procedure: which endpoint, payload size, data volume, test duration, warm-up, client location, and error handling? We cannot responsibly translate “5 requests per second” into a server count without measuring per-request cost and defining headroom. Unit 2 is assigned the complete estimation method. Here we are learning to identify missing evidence.

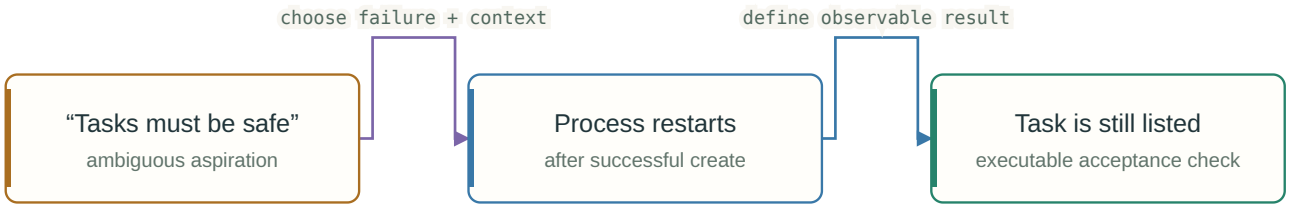


FIG. 1.3.1 **Make a vague quality measurable**

The quality scenario names a stimulus, an environment, and a response. This example concerns process restart; it makes no claim about losing the only disk or an entire region.

Why this view: A precise scope makes a design falsifiable and reveals the assumptions still needing discussion. Author-created illustration.

Table 1.3.1 · A requirement is strongest when it can fail a test

Statement	Problem	More useful scenario
“The app is fast.”	No operation, load, or threshold.	95% of list requests finish within 300 ms under the declared workload.
“Data is safe.”	Failure boundary missing.	Every acknowledged create is readable after process restart, given intact committed storage.
“Users are isolated.”	Actor and action missing.	An authenticated user cannot read or modify another user’s task through any task endpoint.
“The system is maintainable.”	No concrete change scenario.	Add a CLI entry point without copying task validation or SQL into it.

1.3.2 · Name constraints, assumptions, and risks

A **constraint** limits the solution space: use an existing machine, comply with a retention rule, finish within a calendar deadline. An **assumption** is a condition temporarily treated as true: the disk remains intact during process restart; a user does not submit unbounded titles. A **risk** is an uncertain event or condition that could

undermine the outcome: the only host fails, a restore has never been tested, or traffic grows beyond the measured envelope. An assumption can itself become a risk when there is little evidence for it.

A **trade-off** is a choice among competing consequences. Local embedded storage simplifies operations but ties data placement to a host and limits some multi-instance deployment choices. A separate managed database can reduce some host coupling while adding network dependencies, cost, access controls, and migration work. Neither label alone establishes which choice is appropriate.

Architectural characteristics, often called **quality attributes**, describe properties such as performance, reliability, security, maintainability, and operability. Frameworks organize them differently. For example, the AWS Well-Architected framework offers a set of review lenses. Those are engineering guidance, not a theorem guaranteeing that an architecture is correct. Our cloud-neutral design uses such lenses to ask questions about an explicit workload.

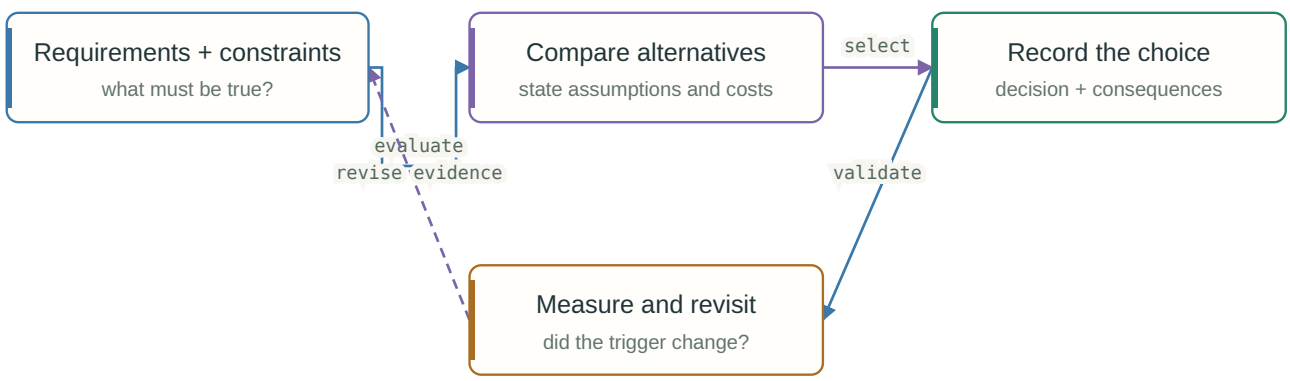


FIG. 1.3.2 **A decision is a testable hypothesis**

An ADR records why a choice fits current evidence and when to revisit it. The feedback dependency returns changed evidence to requirements analysis.

Why this view: It keeps a previously sensible choice from becoming an unquestioned permanent rule. Author-created illustration.

1.3.3 · Feasibility first, preference second

Consider three candidates: memory-only storage, an embedded transactional store, and a separate database service. The first fails the restart requirement, so remove it before scoring preferences. A weighted decision matrix can help expose the remaining disagreements, but cannot prove an optimum. Scores are judgments; weights represent priorities, and their uncertainty matters.

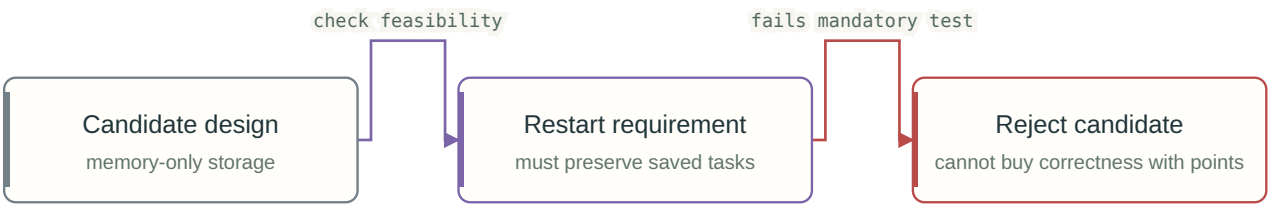


FIG. 1.3.3 **A hard constraint comes before a preference score**

An in-memory design is infeasible for the chosen restart promise, however cheap or quick it is. Weighted scores compare only candidates that pass the hard constraints.

Why this view: It distinguishes a preference trade-off from violating an invariant. Author-created illustration.

Table 1.3.2 · Illustrative design judgments, 1 = poor fit, 5 = strong fit

Candidate	Restart promise	Simplicity	Multi-instance growth	Operational independence from app host
Memory only	Fails	5	1	1
Embedded store	Passes under stated assumptions	5	2	1
Separate database	Passes with appropriate commit settings	2	4	4

We score fit to the stated scenario, not product performance. A separate database does not automatically deliver availability, and an embedded store is not automatically slow. The labels say little without configuration, access patterns, operational ownership, and failure tests.

THE WORKBENCH · AUTHOR-CREATED MODEL

Change the priorities, inspect the decision

Weights (simplicity, growth, host independence): 5, 1, 1.



Embedded store ranks higher under the selected preferences. The scores do not establish throughput or availability.

Memory only: rejected by mandatory restart persistence.

Illustrative fit scores from Table 1.3.2. Restart persistence is mandatory. Preference weights never override feasibility.

With weights 5 for simplicity and 1 each for growth and host independence, the normalized embedded score is `(5×5 + 1×2 + 1×1) / 7 = 4`. The separate-database score is `(5×2 + 1×4 + 1×4) / 7 ≈ 2.57`. Changing priorities changes the ranking. It does not establish that a candidate meets a new hard requirement. If the requirement becomes “continue through loss of the application host,” the deployment and storage failure domains must be reconsidered explicitly.

1.3.4 · Write down why the choice makes sense

An **Architecture Decision Record (ADR)** captures one consequential choice and its rationale. A concise useful record includes context, status, the decision, consequences, and alternatives. Preserve the old rationale when a later decision supersedes it. Michael Nygard’s original ADR article describes this lightweight practice. The record below is an original example, not a universal template.

ADR 001 · ACCEPTED FOR THE TEACHING SCENARIO

Commit tasks in a local transactional store

Context

One small team, one host, low assumed traffic, process-restart persistence, no requirement for host-failure continuity.

Decision

Keep one application process with explicit modules. Use a local embedded database. Commit a task before reporting successful creation.

Alternatives

Memory-only state fails the restart promise. A separate database is feasible but adds an operating dependency that current evidence does not require.

Consequences

We gain a defined local commit boundary and simple deployment. Host or disk loss remains a service and data risk. Backups, restore tests, authorization, and monitoring remain necessary work.

Revisit when

Independent application replicas become necessary, measured write contention exceeds the latency budget, or the business requires recovery from loss of the host.

Evidence to collect

Restart tests, storage failure tests, a restore drill, and representative latency measurements.

The record avoids claiming that a database choice solves operations. It explicitly identifies the remaining work. “Revisit at 10,000 users” would be weaker than a trigger based on a measured bottleneck or a changed failure requirement: user count alone does not determine traffic, data size, or contention.

1.3.5 · Distinguish quality properties before combining them

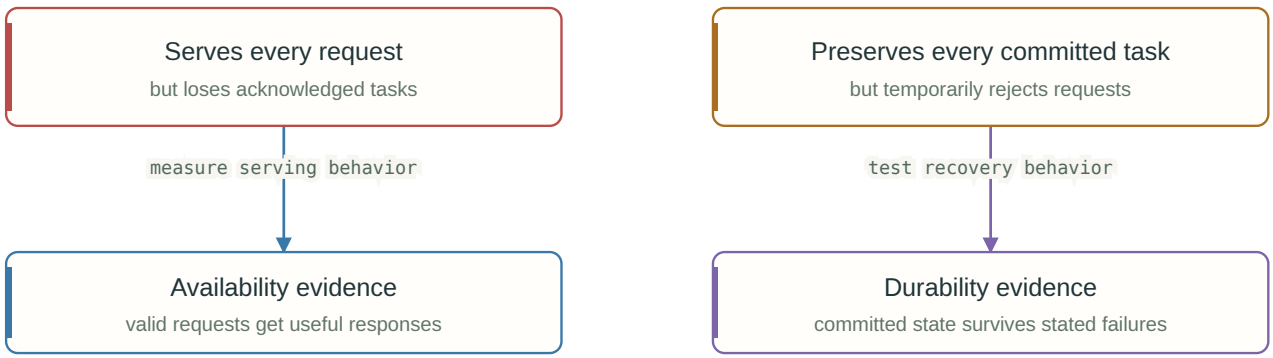


FIG. 1.3.4 **Healthy traffic does not prove durable data**

Availability and durability require different observations. A fully reachable service can lose data; a temporarily unreachable store can retain all committed data.

Why this view: Separate evidence prevents an uptime dashboard from being treated as proof of data safety. Author-created illustration.

Availability concerns whether the service can provide the required response when called. Define what counts as an eligible request and a successful result before measuring it. **Durability** concerns the persistence of committed data under a stated failure model. **Reliability** is often used more broadly for correct service over time; usage varies, so state the operational definition. **Consistency** concerns the allowed observations of state across operations or copies and needs a precise contract in distributed settings.

A replica may increase read capacity while lagging behind writes. A service may return successful HTTP responses while silently discarding tasks. A store may reject requests during recovery yet preserve every committed row. These outcomes answer different questions. Do not use “highly available,” “reliable,” and “durable” interchangeably in a review.

▼ Go deeper · The simplest sufficient design is conditional

“Simple” is not identical to the fewest boxes. One enormous function may have little infrastructure but complicated reasoning. A managed database may add a network hop while simplifying backups and operational ownership for a particular team. Count the complexity that people must understand, operate, and recover, not only the components in a diagram.

"Sufficient" means the design meets the agreed contract with evidence proportionate to its risk. It is not permission to ignore a security requirement because traffic is low. Nor is it a demand to anticipate every hypothetical future feature. Isolate likely changes where the cost is modest, record the uncertainty, and revisit when the evidence changes.

MISCONCEPTION

A weighted matrix cannot trade away a mandatory correctness property. Reject infeasible candidates first. Use scores to discuss preferences among feasible candidates, and investigate close or assumption-sensitive results.

FIELD NOTE

A good ADR can become more useful after it is superseded: it explains which assumption changed. That helps later engineers avoid reintroducing the same unsuitable choice under a different name.

Carry forward: Turn a desire into an observable scenario. Check hard constraints, compare costs, record assumptions, and define what evidence would change the decision.

Before continuing: What observation would justify revisiting ADR 001 even if the system still had only 20 users?

Sources & further reading

Read from introductory guidance to the primary or implementation detail relevant to this chapter. Links checked 21 October 2026.

1. Documenting Architecture Decisions — Michael Nygard, 2011 ↗

   Original practice article. ADR structure and decision history.

   `Link checked · 2026-10-21`

2. AWS Well-Architected: the six pillars ↗

   Official guidance · accessed 21 Oct 2026. Review lenses, not correctness guarantees.

   `Link checked · 2026-10-21`

One application, many architectural views

Prerequisite: Chapter 1.3 4 original figures · 1 workbench

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / infrastructure

Gray: external / inactive Solid arrow: action / data · dashed: dependency or zoom · dotted: acknowledgment

A reviewer points at the architecture diagram and asks, “Are those three boxes three servers?” The author meant three modules. Another arrow means a compile-time dependency; the reviewer thinks it is an HTTP call.

A useful diagram answers a specific question and tells the reader how to interpret every boundary and arrow.

1.4.1 · Choose the view for the question

A static structure view shows elements and relationships. A sequence view shows ordered interactions in a scenario. A deployment view places executable artifacts and data on infrastructure. They can describe the same application while revealing different facts. A single diagram containing users, methods, disks, regions, and classes usually mixes too many levels to reason clearly.

The C4 model’s original documentation defines four static abstraction levels: system context, containers, components, and code. Use the levels that help the audience. Sequence or dynamic views and deployment views provide additional perspectives rather than extra steps in that four-level hierarchy.

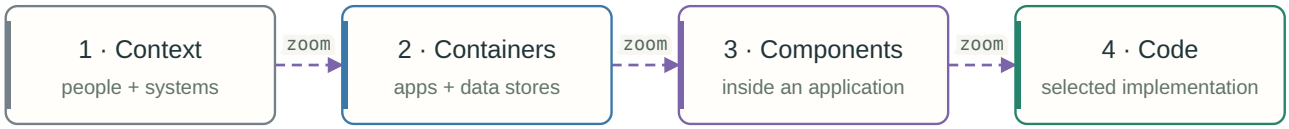


FIG. 1.4.1 **C4 changes the level of detail**

The dashed arrows mean a change in abstraction, not a runtime call. C4 containers are applications or data stores; they are not synonymous with operating-system containers.

Why this view: Choose the view that answers a real reader question, rather than drawing all four levels automatically. Author-created illustration.

Table 1.4.1 · One system, different questions

View	Question	Our example	Detail deliberately omitted
Context	Who uses it, and what systems surround it?	A team member uses the task system.	SQL tables, method signatures, host layout.
Container	What applications and data stores cooperate?	Browser client, server application, local data store.	Individual service objects and classes.
Component	What responsibilities collaborate inside an application?	Input adapter, task service, storage adapter.	Physical rack or virtual-machine placement.
Code	How is a selected component implemented?	Task data type, service functions, repository protocol.	Unrelated framework classes.
Sequence	What happens in this ordered scenario?	Validate, insert, commit, respond; then lose reply.	Full infrastructure inventory.
Deployment	Where do execution and data live?	One host runs the app and holds the database file.	Unneeded business rule internals.

In C4 terminology a **container** can be an application or data store. It is not necessarily a Docker container. The project's container definition makes this distinction explicit. Similarly, a box marked "TaskRepository" in

a component view need not denote a database service or an independent process.

THE WORKBENCH · AUTHOR-CREATED MODEL

Ask the same application four questions

Who uses the system?

Team member → uses → Task system

The system boundary includes responsibility for task behavior. Internal deployment is intentionally hidden.

Four views of the same teaching system. A zoom relationship is not a network call. Figure 1.4.1 and Table 1.4.1 provide the complete static version.

1.4.2 · Progress from a script to a small deployed application

Let us make the evolution concrete. Version A is the in-memory script. It is sufficient for ephemeral session notes. Version B keeps the same local interaction but separates task rules from storage and commits to an embedded database. Version C introduces an HTTP adapter because multiple people need remote access. The task rule remains in the service. The storage adapter remains responsible for the commit.

Introducing remote access changes the obligations. The input is untrusted, transport can fail, requests can repeat, identities matter, and requests may overlap. If tasks are private to their owners, every task operation needs an authorization check tied to the authenticated identity. Hiding an identifier in the UI is not access control. That requirement must be enforced server-side for list, read, completion, and deletion. Unit 18 is the planned threat-modeling and isolation treatment; the simple local Python exercise intentionally handles only one trusted local user.

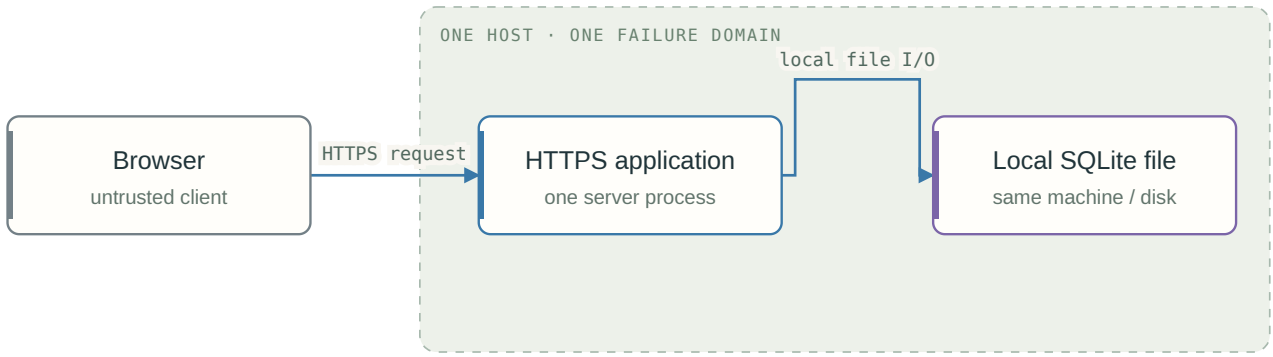


FIG. 1.4.2 **A deployment view locates the single points of failure**

The example has one machine with a local store. Module separation has not created redundancy; host or disk loss can still remove the whole service.

Why this view: Deployment placement exposes correlated failure that logical component boxes can hide. Author-created illustration.

This deployment uses one machine and one local store. It has no redundancy. The browser and application are separated by a trust and network boundary; the service and repository are modules in the same process. The database file resides on the host. Moving the web application to two hosts while keeping a local file on only one does not create a working shared-data design. A change in deployment can invalidate an earlier storage assumption.

1.4.3 · Trace an actual creation request

```
# Illustrative API contract, not a running HTTP server.
POST /tasks
Content-Type: application/json
Body: {"title": "Read"}

1. Authenticate the caller; obtain owner identity from the session.
2. Parse and bound the input; reject invalid syntax or title.
3. Begin a storage transaction.
4. Insert task owned by the authenticated caller.
5. Commit the transaction.
6. Return 201 Created, Location: /tasks/7,
   and {"id": 7, "title": "Read", "done": false}.
```

The response is sent after commit. If validation fails, no task is written. If the transaction fails before commit, no success is sent. If the process crashes after commit but before step 6, the task exists and the caller may see only a disconnection. If the network loses step 6, the application can be completely healthy while the caller still lacks the result. RFC 9110 defines the HTTP 201 response semantics; our commit ordering and duplicate-handling policy are additional application contracts.

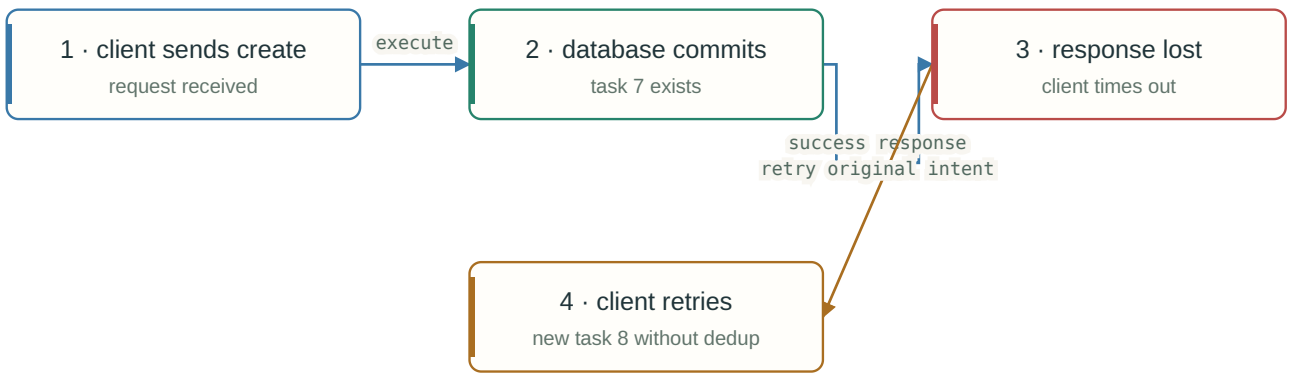


FIG. 1.4.3 **A lost response makes the outcome uncertain**

Commit precedes the lost response. A timeout cannot distinguish this case from a failure before commit. Retrying a create without a deduplication contract can create another task.

Why this view: A sequence explains the ambiguity that a static architecture diagram leaves out. Author-created illustration.

This is the reason sequence diagrams need explicit ordering. Two unlabeled arrows labeled "request/response" would hide the decisive event: the durable commit. The diagram also needs to say what has *not* been implemented. Here, a retry is a new creation. To deduplicate, the client would supply a stable operation identity; the server would bind it to the request and committed result with an appropriate atomicity and retention policy. We defer that full mechanism to 8.2 and the payment case study.

1.4.4 · Make arrows and state copies explicit

Use a solid blue arrow for the normal request or data flow, a dashed purple arrow for a source dependency or a zoom relationship, and a dotted arrow for an acknowledgment when that is the chosen legend. Label the action: "HTTPS create," "commit task," "implements protocol." Place a legend alongside a view if color or line style carries meaning. A red boundary alone is inadequate; label the failure or trust property in words.

State locations deserve special care. The browser may display a local snapshot. The server may hold an in-memory object. The database holds the committed record. Those are different copies with different lifetimes. After another user changes a record, the browser's displayed copy can be stale even if the

database is behaving correctly. A diagram that labels all three copies simply “task data” conceals the freshness contract.

<details>
<summary>Go deeper · A diagram review with four adversarial questions</summary>

1. **Kill one box.** If the host fails, which other boxes disappear with it? Logical boundaries do not imply independent failure.
2. **Delay one arrow.** Which caller waits, for how long, and who owns cancellation? A synchronous dependency consumes a caller’s time budget.
3. **Repeat one message.** Does the state change twice? If not, where is duplicate identity stored and how long is it retained?
4. **Change one representation.** Which collaborators must deploy together? An apparently modular diagram can hide shared schemas or ordering assumptions.

These are tests of the explanation, not complete failure proofs. Later units add transaction anomalies, replication histories, security threats, and workload tests to the same review habit.
</details>

1.4.5 · Evolution requires a reason and a review

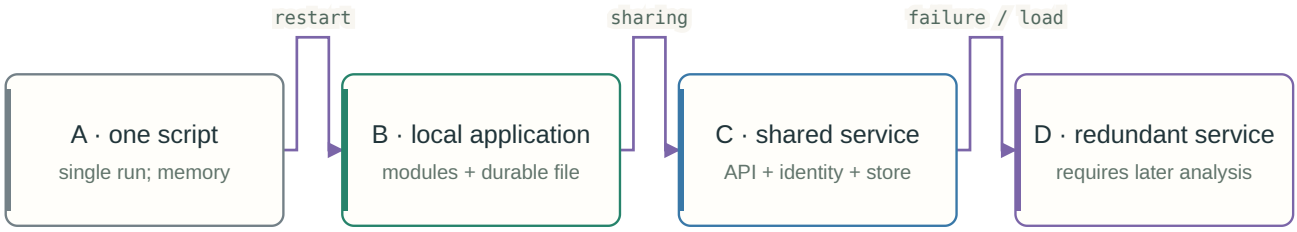


FIG. 1.4.4 **Architecture evolves when an assumption fails**

Persistence addresses restart. An API addresses shared remote access. Redundancy and data distribution require their own failure, correctness, and load analysis; they are not a free final step.

Why this view: The sequence is a set of justified changes, not a universal maturity ladder. Author-created illustration.

Suppose the shared service eventually misses its latency objective. First locate the bottleneck. If the database is waiting on one slow query, adding application replicas can increase pressure without improving

completion rate. If CPU work dominates and the state path has headroom, more application workers may help. If the availability requirement changes to survive a host failure, the design must address both execution and state placement. “Add servers” is an action, not an argument.

Our first design review is therefore conditional. The current design is reasonable for its declared small, single-host scenario. It remains exposed to host loss, disk loss, untested restores, unknown concurrent-load behavior, and repeated creates after lost replies. The next production stages must either resolve those exposures or explicitly accept them against requirements. A list of famous infrastructure products would not supply that reasoning.

> MISCONCEPTION
>
> A module diagram is not a deployment topology. Three components inside a single process still share memory pressure, process crashes, and the host’s failure domain.

> FIELD NOTE
>
> The most useful annotation on an architecture drawing is often a short assumption: “one writer,” “commit before response,” “same host,” or “snapshot may be stale.” It tells a future reviewer exactly where to push.

Carry forward: Use a structure view to assign responsibility, a sequence to expose ordering, and a deployment view to locate failures. Check that the three tell the same story.

Before continuing: Which diagram would reveal an accidental success-before-commit bug? Which would reveal that the application and its backup share the same disk?

Sources & further reading

Read from introductory guidance to the primary or implementation detail relevant to this chapter. Links checked 21 October 2026.

1. C4: diagrams ↗

   Original model documentation. Formal diagram levels and supporting views.

   `Link checked · 2026-10-21`

2. C4: container abstraction ↗

   Original model documentation. Meaning of container.

   Link checked · 2026-10-21

3. RFC 9110: HTTP Semantics, June 2022 ↗

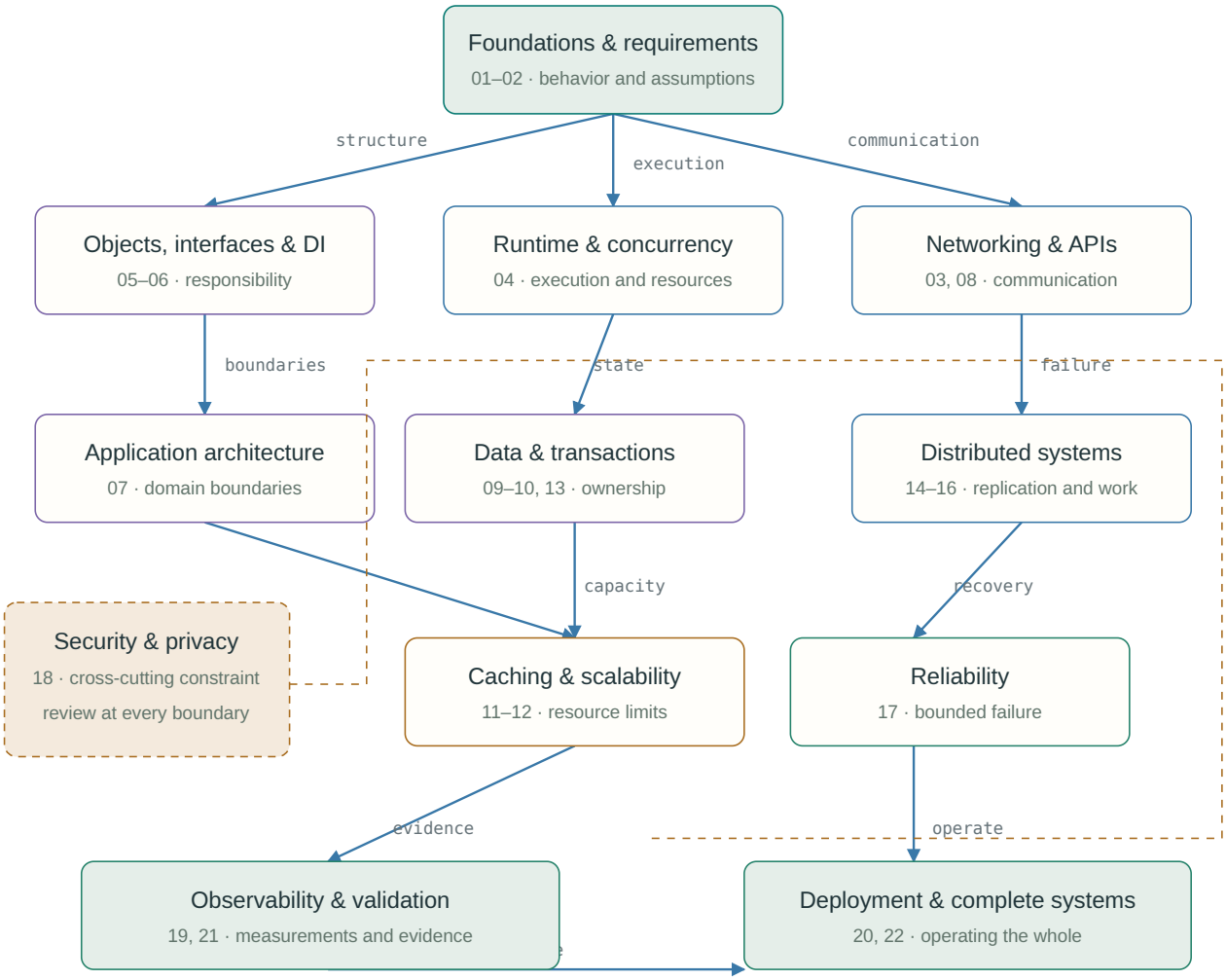
   Original standard. HTTP semantics; application commit policy is separate.

   Link checked · 2026-10-21

HOW THE IDEAS CONNECT

A map of the territory

Read the arrows as prerequisite relationships. Security and operations also constrain designs throughout the map.



Simplified concept spine; exact unit and chapter prerequisites appear in Reading paths.

FIG. M.1 **The ideas form a dependency network.**

A simplified prerequisite spine connects low-level structure to storage, networking, distributed behavior, and production engineering. Security constrains the entire design.

Why this view: it shows why a replication or scaling mechanism needs earlier reasoning about state and execution. Author-created illustration.

Solid arrow = prerequisite · dashed line = a cross-cutting constraint · author-created illustration

Why these connections matter

Interfaces make ownership explicit. Data models determine which invariants transactions must protect. Concurrency makes execution order observable. Networks introduce partial failure. Replication combines those concerns; reliability, security, deployment, and observability determine whether the resulting system can be operated.

BEGIN WHERE YOU ARE

Dependency-aware reading paths

Routes select a goal; chapter prerequisites still apply. Planned units are included as an editorial path, not as available lessons.

First pass · from program to service

01 → 02 → 03 → 04 → 05 → 06 → 07 → 08 → 09 → 10

Data and scale

01 → 02 → 03 → 04 → 08 → 09 → 10 → 11 → 12 → 13

Distributed correctness

01 → 03 → 04 → 09 → 10 → 14 → 15 → 16 → 17

Production engineering

01 → 02 → 03 → 04 → 05 → 06 → 07 → 08 → 09 → 10 → 11 → 12 → 14 → 15 → 16 → 17 → 18 → 19 → 20 → 21

The complete prerequisite graph

Unit	Direct prerequisites	Reason to read next
1 · Introduction to System Design	Basic programming	A system is a promise
2 · Requirements, Estimation, and Engineering Trade-offs	1	Turn user journeys into workloads
3 · Networking Foundations for System Design	1	Addresses, sockets & transports
4 · Runtime, Processes, Threads, and Concurrency	1	The runtime owns your resources
5 · Object-Oriented and Modular Software Design	1	Model behavior with objects and values
6 · Design Patterns, Dependency Injection, and Object Lifecycles	5	Dependencies, inversion & composition
7 · Application Architecture and Domain-Driven Design	5, 6	Layers, monoliths & ports
8 · API Design and Communication Contracts	1, 3	Model resources and messages
9 · Data Models and Storage Technologies	1	Storage promises & database families
10 · Database Internals, Transactions, and Concurrency Control	4, 9	Pages, indexes & query plans
11 · Caching and Content Delivery	3, 9	Locality and cache placement
12 · Scalability, Load Balancing, and Data Distribution	2, 3, 9, 10, 11	Scale compute & route requests
13 · File Systems, Search Infrastructure, and Data Pipelines	8, 9, 10	Upload bytes without losing identity
14 · Distributed Systems Fundamentals	3, 4, 10	Partial failure & the meaning of time
15 · Replication, Leader Election, and Consensus	14	Choose a replication model

Unit	Direct prerequisites	Reason to read next
16 · Message Queues, Events, and Distributed Workflows	8, 10, 14, 15	Queues, logs & delivery boundaries
17 · Reliability, Fault Tolerance, and Disaster Recovery	2, 12, 15, 16	Failure domains & redundancy
18 · Security Architecture and Privacy	3, 7, 8, 9	Threats, identity & authorization
19 · Observability, Monitoring, and Incident Engineering	2, 3, 4, 10	Observe behavior with three signals
20 · Deployment, Cloud Infrastructure, and Platform Architecture	3, 7, 17, 18, 19	Environments, VMs & containers
21 · Performance, Cost Optimization, and System Validation	2, 4, 10, 12, 17, 19, 20	Budgets, bottlenecks & honest benchmarks
22 · End-to-End Design, Architecture Reviews, and Evolution	1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21	URL Shortener

Recurring systems & case-study progression

The to-do application establishes boundaries and commit timing in Unit 1. Payments connect refactoring (5), API retries (8), workflow compensation (16), provider outages (17), and the final ledger design (22.9). Ticket booking connects transaction races (10) to reservations and expiry (22.8). Document processing connects ingestion and lineage (13) to durable orchestration and human review (22.12). URL shortening connects measured scaling (12) to complete design review (22.1).

UNIT 01 / ASSESSMENT

Reason about the design

Write an answer before opening the separate solutions. These questions test assumptions and failure histories.

CONCEPTUAL / C1

A program reports “saved” after adding a task to a map. Which contract can it satisfy, and which contract does it fail?

Read the explanation →

CONCEPTUAL / C2

How does a method signature differ from a behavioral contract?

Read the explanation →

CONCEPTUAL / C3

Does low coupling require removing all dependencies?

Read the explanation →

CONCEPTUAL / C4

Are HLD, LLD, and deployment separate formal categories of software?

Read the explanation →

CONCEPTUAL / C5

How can a system be available but lose data, or durable but temporarily unavailable?

Read the explanation →

SCENARIO / S1

The database commits task 7; the success response is lost; the client retries. Can the client safely assume the first operation failed?

Read the explanation →

SCENARIO / S2

Two adapters implement the same 80-character title limit. It changes to 200, but only one adapter is edited. What design change addresses the cause?

Read the explanation →

SCENARIO / S3

The team adds a second application host, each with its own local task database. What new failure has been introduced?

Read the explanation →

DESIGN / D1

Review this design: HTTP handler → shared dictionary → async file writer; the handler returns success immediately. Draw the failure and propose the smallest change that meets the restart promise.

Read the explanation →

IMPLEMENTATION / I1

Run examples/todo.py. Add a failing repository and prove that a failed insert never returns a Task. Then reopen the real store and prove a successful task remains.

Read the explanation →

THOUGHT EXPERIMENT / T1

The business now requires “no acknowledged task lost after the only disk is destroyed.” Can the current ADR stand? What evidence would a replacement need?

Read the explanation →

UNIT 01 / EXPLAINED ANSWERS

Solutions & reasoning

Each explanation identifies the relevant assumption and why a tempting alternative fails.

C1

A program reports “saved” after adding a task to a map. Which contract can it satisfy, and which contract does it fail?

It can satisfy a session-only create-and-list contract under sequential execution. It fails the chosen process-restart persistence promise because the next process constructs a new map. Renaming the map a repository or putting it behind an interface changes organization, not persistence.

← Return to question

C2

How does a method signature differ from a behavioral contract?

The signature identifies inputs and outputs at the type level. The contract also states validation, side effects, errors, commit timing, retry behavior, and concurrency or lifetime assumptions. Two insert(title) methods can be incompatible if one returns before commit and the other promises persistence.

← Return to question

C 3

Does low coupling require removing all dependencies?

No. Useful behavior requires collaboration. Reduce unnecessary knowledge and make necessary obligations explicit. One dependency on a global mutable schema can cause more change propagation than several narrow stable interfaces. An interface that hides neither knowledge nor responsibility may merely add indirection.

← Return to question

C 4

Are HLD, LLD, and deployment separate formal categories of software?

They are common organizing viewpoints with community-dependent boundaries. HLD and LLD describe consequential structure at different detail; deployment locates runtime artifacts. They must agree about responsibility, ordering, and placement. A class diagram does not establish host redundancy.

← Return to question

C 5

How can a system be available but lose data, or durable but temporarily unavailable?

An application can keep accepting requests while acknowledging only volatile state, then lose that state after restart. A store can retain every committed row while refusing requests during recovery. Uptime measurements alone do not demonstrate durability, and retained data alone does not establish serving availability.

← Return to question

S 1

The database commits task 7; the success response is lost; the client retries. Can the client safely assume the first operation failed?

No. The timeout provides insufficient evidence about commit. A second unkeyed create may produce task 8. To bind repeated attempts to one intent, define a stable operation key, request fingerprint, atomic result storage, and retention behavior. Blind retries and a label saying “exactly once” do not supply that contract.

← Return to question

S 2

Two adapters implement the same 80-character title limit. It changes to 200, but only one adapter is edited. What design change addresses the cause?

Put the authoritative rule in the shared task service and have both adapters call it. Adapters still validate transport syntax and bound payloads. Repeating the edit in both adapters repairs this instance but preserves the duplication risk. Moving both adapters into different services would make coordinated changes harder.

← Return to question

S 3

The team adds a second application host, each with its own local task database. What new failure has been introduced?

Requests routed to different hosts can observe different task sets. Local persistence is not shared state or replication. The prior single-host assumption has changed, so review data placement, consistency, identity generation, and failure behavior. A load balancer cannot merge the two databases.

← Return to question

D1

Review this design: HTTP handler → shared dictionary → async file writer; the handler returns success immediately. Draw the failure and propose the smallest change that meets the restart promise.

Failure trace: mutate dictionary, return success, crash before async writer commits; restart loses an acknowledged task. The minimal correction is to wait for an appropriate durable commit before reporting success, propagating write failures. A transaction-aware local repository is one feasible implementation. A queue alone only moves the durability boundary; its acknowledgment semantics would still need justification. Keep the modules in one deployment unless another requirement calls for distribution.

← Return to question

I1

Run examples/todo.py. Add a failing repository and prove that a failed insert never returns a Task. Then reopen the real store and prove a successful task remains.

The service must propagate the repository exception; it must not manufacture a success object after failure. Use a repository whose insert raises OSError and assert create raises. With SQLiteTasks, create a task, close the connection, open a new connection to the same temporary path, and compare the stored record. Invalid titles must not insert any row. These tests exercise different promises; an in-memory double cannot demonstrate real persistence.

← Return to question

T 1

The business now requires “no acknowledged task lost after the only disk is destroyed.” Can the current ADR stand? What evidence would a replacement need?

The current ADR explicitly assumes intact committed storage, so its design is insufficient for the new requirement. Consider independent durable copies, acknowledgment rules, geographic or device failure correlation, backups versus live replication, and tested recovery. Asynchronous copying can lose recently acknowledged writes; copying corruption can preserve the wrong state. A replacement must specify the failure model and acceptable recovery behavior, then test its claims. More replicas without acknowledgment and independence rules do not establish the guarantee.

← Return to question

Next production stage · Requirements, estimation, and trade-offs →

THE REFERENCE DESK

A searchable glossary

Compact definitions support reading; a link to a planned unit identifies where the full treatment belongs.

44 of 44 terms

ADR

Architecture Decision Record: one consequential choice, its context, rationale, status, and consequences.

Read the lesson →

Abstraction

A behavioral view that hides details a collaborator does not need, while retaining relevant obligations.

Read the lesson →

Application architecture

The organization and interaction of application responsibilities and their dependency rules.

Planned treatment →

Availability

Ability to provide the specified useful response to eligible requests under stated conditions.

Read the lesson →

Backpressure

A mechanism that communicates or enforces downstream capacity limits on upstream work.

Planned treatment →

Boundary

A chosen division of responsibility, trust, execution, or representation. State which kind.

Read the lesson →

CAP

Under a network partition, the theorem rules out simultaneously guaranteeing its formal consistency and availability properties. It is not a general “pick any two” menu.

Planned treatment →

Cohesion

How closely a module’s elements contribute to a focused responsibility.

Read the lesson →

Component

A unit of responsibility in a chosen design view; not necessarily a process or service.

Read the lesson →

Composition root

The part of the program that constructs and connects concrete collaborators.

Read the lesson →

Concurrency

Multiple activities in progress with potentially interleaved execution; parallel execution is not required.

Planned treatment →

Consistency model

A contract specifying which observations of shared or replicated state are allowed.

Planned treatment →

Container (C4)

An application or data store in C4 terminology; not necessarily an OS container.

Read the lesson →

Contract

An interface’s behavioral obligations, including inputs, outcomes, errors, state effects, and timing assumptions.

Read the lesson →

Coupling

What an element depends on or must know about another element.

Read the lesson →

Dependency injection

Supplying collaborators to an object or function rather than having it locate or construct them internally.

Planned treatment →

Durability

Persistence of committed state under an explicitly stated set of failure assumptions.

Read the lesson →

Encapsulation

Controlling access to a representation so behavior and invariants cannot be casually bypassed.

Read the lesson →

Error budget

The allowed unreliability implied by a chosen SLO over its measurement window.

Planned treatment →

Eventual consistency

A convergence property under the model's assumptions; it alone gives neither an immediate freshness bound nor a complete conflict policy.

Planned treatment →

Exactly-once processing

A scoped guarantee about effects or records within a defined system boundary and assumptions. It does not automatically include arbitrary external side effects.

Planned treatment →

Failure domain

Elements exposed to a common cause of failure, such as a process, host, disk, or region.

Planned treatment →

HLD

High-Level Design: a common view of major components, state placement, communication, and system qualities.

Read the lesson →

Idempotency

Repeated application of an operation has the same relevant effect as one application, under its defined identity and scope.

Planned treatment →

Interface	The operations and data forms exposed to a collaborator; behavior needs an accompanying contract. Read the lesson →
Invariant	A condition required to hold in every valid system state. Read the lesson →
LLD	Low-Level Design: a common view of internal modules, types, interfaces, algorithms, and collaboration. Read the lesson →
Latency	Elapsed time for an operation under a specified measurement boundary. Planned treatment →
Linearizability	A correctness condition in which operations appear atomic and respect real-time precedence. Planned treatment →
MVCC	Multiversion concurrency control: retaining versions to help coordinate concurrent database access. Exact visibility and isolation depend on implementation. Planned treatment →
Modularity	Separating responsibilities with explicit interfaces and controlled dependencies. Read the lesson →
Nonfunctional requirement	A constraint on the quality or conditions of system behavior, such as latency or persistence. Read the lesson →
Quorum	A set of participants whose responses meet a protocol’s decision rule. Intersection alone does not establish all read or write correctness properties. Planned treatment →

RPO

Recovery Point Objective: an objective for the amount of data loss measured in time.

Planned treatment →

RTO

Recovery Time Objective: an objective for restoring service after disruption.

Planned treatment →

Repository

A component that mediates access to a data model through an explicit contract. Its transaction ownership must be specified.

Read the lesson →

SLI / SLO / SLA

Indicator: measured behavior. Objective: target for that behavior. Agreement: an explicit service agreement, potentially with consequences.

Planned treatment →

Saga

A coordinated sequence of local transactions with compensating actions; compensation need not be a perfect inverse.

Planned treatment →

Serializability

A transactional correctness condition requiring equivalence to some serial execution; real-time order is a separate issue.

Planned treatment →

State

Information whose current value influences subsequent system behavior.

Read the lesson →

Throughput

Completed work per unit time under a specified workload and success definition.

Planned treatment →

Trade-off

A choice among competing consequences evaluated against requirements and assumptions.

Read the lesson →

Transactional outbox

Recording a state change and publication intent in one local transaction, then relaying the intent; duplicate delivery still requires handling.

Planned treatment →

Trust boundary

A point across which identity, privilege, or confidence in input changes.

Planned treatment →

No terms match that search.

THE VISUAL INDEX

Every diagram has a job

Sixteen original figures accompany the four produced chapters. Further figures remain commissions in the chapter blueprint.

FIG. 1.1.1

The boundary follows the promise

It separates the user-visible obligation from the process that happens to run the code.

FIG. 1.1.2

Data becomes state through a transition

A concrete transition exposes both the state owner and the effect that a retry could duplicate.

FIG. 1.1.3

HLD and LLD ask connected questions

Both views must agree about who commits state and who reports success.

FIG. 1.1.4

From a working demo to a recoverable write

The improvement is tied to a reproducible failure rather than an arbitrary desire for more components.

FIG. 1.2.1

An innocent feature crosses every responsibility

Dependency arrows identify the concrete knowledge that makes a change spread.

FIG. 1.2.2

Separate responsibilities inside one process

A module boundary can reduce change propagation without adding a network boundary.

FIG. 1.2.3

Dependency direction differs from execution direction

It prevents the common mistake of interpreting every diagram arrow as a network request.

FIG. 1.2.4

Abstraction narrows the question at each layer

Abstraction reduces what a caller must know; it cannot remove physical dependencies.

FIG. 1.3.1

Make a vague quality measurable

A precise scope makes a design falsifiable and reveals the assumptions still needing discussion.

FIG. 1.3.2

A decision is a testable hypothesis

It keeps a previously sensible choice from becoming an unquestioned permanent rule.

FIG. 1.3.3

A hard constraint comes before a preference score

It distinguishes a preference trade-off from violating an invariant.

FIG. 1.3.4

Healthy traffic does not prove durable data

Separate evidence prevents an uptime dashboard from being treated as proof of data safety.

FIG. 1.4.1

C4 changes the level of detail

Choose the view that answers a real reader question, rather than drawing all four levels automatically.

FIG. 1.4.2

A deployment view locates the single points of failure

Deployment placement exposes correlated failure that logical component boxes can hide.

FIG. 1.4.3

A lost response makes the outcome uncertain

A sequence explains the ambiguity that a static architecture diagram leaves out.

FIG. 1.4.4

Architecture evolves when an assumption fails

The sequence is a set of justified changes, not a universal maturity ladder.

THE FORMULA REFERENCE

Symbols with assumptions

Two models are introduced in Unit 1. Later formulas are indexed with their conditions and planned teaching destination.

INTRODUCED

State transition

```
T(S, c) → (S′, result)
```

Identify valid state changes and preserved invariants. This abstracts away implementation and failures.

Lesson →

INTRODUCED

Weighted preference

```
score(a) = Σ wᵢ sᵢ(a) / Σ wᵢ
```

Use nonnegative weights with positive total. Compare only feasible designs. Scores are illustrative judgments, not benchmark results.

Lesson →

PLANNED

Request rate

```
λ = daily users × requests per user per day / 86,400
```

An average workload estimate. Peak factors must be assumed or measured separately.

Planned derivation →

PLANNED

Little’s Law

```
L = λ W
```

Long-run stable averages with consistent boundaries and units. It is not a universal p99 prediction.

Planned derivation →

PLANNED

Amdahl’s Law

```
speedup(N) = 1 / [(1 − p) + p/N]
```

Idealized fixed workload and parallel fraction p; excludes extra coordination overhead unless modeled.

Planned derivation →

PLANNED

Storage growth

```
bytes/day = records/day × bytes/record × retained copies
```

Account separately for indexes, compression, retention, metadata, and headroom.

Planned derivation →

PLANNED

Error allowance

```
allowed bad events = (1 − target fraction) × eligible events
```

Valid for an event-based SLO; time-based objectives need their own denominator.

Planned derivation →

PLANNED

Token refill

```
tokens′ = min(capacity, tokens + refill_rate × elapsed)
```

Admission consumes tokens atomically. Distributed clocks and state ownership need additional design.

Planned derivation →

PLANNED

Quorum intersection

```
R + W > N
```

An intersection condition for reads and writes on a fixed replica set. Not by itself a proof of linearizability.

Planned derivation →

PLANNED

Cost per outcome

```
cost / successful business outcomes
```

Define accounting period and include capacity, storage, networking, telemetry, and operational work where relevant.

Planned derivation →

THE PATTERN INDEX

Start with the problem, then the pattern

This is a destination index. The full pattern lessons are planned; names alone do not constitute coverage.

6.1 · Dependencies, inversion & composition

Dependency Injection | Composition Root | Inversion of Control

Planned · mechanism, miniature implementation, failure behavior, and alternatives required.

6.4 · Creational patterns

Factory Method | Abstract Factory | Builder | Prototype | Singleton

Planned · mechanism, miniature implementation, failure behavior, and alternatives required.

6.5 · Structural patterns

Adapter | Bridge | Composite | Decorator | Facade | Flyweight | Proxy

Planned · mechanism, miniature implementation, failure behavior, and alternatives required.

6.6 · Behavioral patterns

Strategy | Observer | Command | State | Chain of Responsibility | Template Method | Iterator | Mediator | Memento | Visitor

Planned · mechanism, miniature implementation, failure behavior, and alternatives required.

6.7 · Enterprise patterns & boundaries

Repository | Unit of Work | Specification | CQRS

Planned · mechanism, miniature implementation, failure behavior, and alternatives required.

Application Architecture and Domain-Driven Design

Modular Monolith | Layered Architecture | Hexagonal Architecture | Clean Architecture | Anti-corruption Layer

Planned · mechanism, miniature implementation, failure behavior, and alternatives required.

Caching and Content Delivery

Cache-aside | Read-through | Write-through | Write-behind | Stale-while-revalidate

Planned · mechanism, miniature implementation, failure behavior, and alternatives required.

Message Queues, Events, and Distributed Workflows

Transactional Outbox | Saga | Event Sourcing | Orchestration | Choreography

Planned · mechanism, miniature implementation, failure behavior, and alternatives required.

Reliability, Fault Tolerance, and Disaster Recovery

Circuit Breaker | Bulkhead | Retry with Jitter | Load Shedding

Planned · mechanism, miniature implementation, failure behavior, and alternatives required.

Deployment, Cloud Infrastructure, and Platform Architecture

Rolling Deployment | Blue-green Deployment | Canary Release

Planned · mechanism, miniature implementation, failure behavior, and alternatives required.

THE READING ROOM

Sources, close to the claim

The book distinguishes original research, formal specifications, official documentation, and engineering guidance. No company architecture or benchmark is invented here.

“Link checked” means that the source was opened and inspected for the stated use on 21 October 2026. It does not mean all future chapters have been verified. Supplied references not yet checked remain explicitly pending. Version-sensitive documentation must be rechecked during chapter production.

46 of 46 sources

SUPPLIED STARTING REFERENCE

Microsoft Architecture Center: application fundamentals ↗

Verification pending before chapter production

Link verification pending

SUPPLIED STARTING REFERENCE

System Design Primer — Donne Martin ↗

Verification pending before chapter production

Link verification pending

OFFICIAL GUIDANCE · ACCESSED 21 OCT 2026

AWS Well-Architected: the six pillars ↗

Review lenses, not correctness guarantees

Link checked · 2026-10-21

ORIGINAL MODEL DOCUMENTATION

C4: diagrams ↗

Formal diagram levels and supporting views

Link checked · 2026-10-21

ORIGINAL MODEL DOCUMENTATION

C4: container abstraction ↗

Meaning of container

Link checked · 2026-10-21

ORIGINAL PRACTICE ARTICLE

Documenting Architecture Decisions — Michael Nygard, 2011 ↗

ADR structure and decision history

Link checked · 2026-10-21

OFFICIAL DOCUMENTATION · ACCESSED 21 OCT 2026

Atomic Commit in SQLite ↗

Commit and operating-system assumptions

Link checked · 2026-10-21

ORIGINAL STANDARD

RFC 9110: HTTP Semantics, June 2022 ↗

HTTP semantics; application commit policy is separate

Link checked · 2026-10-21

SUPPLIED STARTING REFERENCE

The Site Reliability Workbook: contents ↗

Verification pending before chapter production

Link verification pending

SUPPLIED STARTING REFERENCE

MDN: an overview of HTTP ↗

Verification pending before chapter production

Link verification pending

SUPPLIED STARTING REFERENCE

Cloudflare Learning Center: content delivery networks ↗

Verification pending before chapter production

Link verification pending

SUPPLIED STARTING REFERENCE

System Design 101 — ByteByteGo ↗

Verification pending before chapter production

Link verification pending

UNIVERSITY TEXTBOOK

Operating Systems: Three Easy Pieces ↗

Runtime, concurrency, and persistence

Link checked · 2026-10-21

SUPPLIED STARTING REFERENCE

The Go memory model ↗

Verification pending before chapter production

Link verification pending

SUPPLIED STARTING REFERENCE

Refactoring Guru: design patterns ↗

Verification pending before chapter production

Link verification pending

SUPPLIED STARTING REFERENCE

Bounded Context — Martin Fowler ↗

Verification pending before chapter production

Link verification pending

ORIGINAL ENGINEERING ARTICLE

Inversion of Control Containers and the Dependency Injection pattern — Martin Fowler, 2004 ↗

Dependency injection and wiring

Link checked · 2026-10-21

SUPPLIED STARTING REFERENCE

Refactoring Guru: the pattern catalog ↗

Verification pending before chapter production

Link verification pending

SUPPLIED STARTING REFERENCE

Microsoft Architecture Center: architecture styles ↗

Verification pending before chapter production

`Link verification pending`

SUPPLIED STARTING REFERENCE

Microservices — James Lewis and Martin Fowler ↗

Verification pending before chapter production

`Link verification pending`

SUPPLIED STARTING REFERENCE

MDN: HTTP documentation ↗

Verification pending before chapter production

`Link verification pending`

SUPPLIED STARTING REFERENCE

Microsoft Architecture Center: application best practices ↗

Verification pending before chapter production

`Link verification pending`

PROJECT DOCUMENTATION

OWASP API Security Top 10, 2023 edition ↗

Threat taxonomy for Unit 18

`Link checked · 2026-10-21`

UNIVERSITY COURSES

Carnegie Mellon Database Group: courses ↗

Database internals reading

`Link checked · 2026-10-21`

SUPPLIED STARTING REFERENCE

CMU 15-445/645: database systems ↗

Verification pending before chapter production

`Link verification pending`

OFFICIAL DOCUMENTATION · ACCESSED 21 OCT 2026

PostgreSQL 18: transaction isolation ↗

Version-specific isolation behavior; recheck current alias before later production

`Link checked · 2026-10-21`

SUPPLIED STARTING REFERENCE

Redis: application patterns — supplied URL redirects ↗

Verification pending before chapter production

`Link verification pending`

SUPPLIED STARTING REFERENCE

Microsoft Architecture Center: cloud design patterns ↗

Verification pending before chapter production

`Link verification pending`

SUPPLIED STARTING REFERENCE

Microsoft Architecture Center ↗

Verification pending before chapter production

Link verification pending

OFFICIAL DOCUMENTATION · VERSION SELECTOR

Apache Kafka documentation ↗

Version-sensitive; specific guarantees must be pinned in Unit 16

Link checked · 2026-10-21

UNIVERSITY COURSE

MIT 6.5840: Distributed Systems, Spring 2026 ↗

Advanced distributed-systems reading

Link checked · 2026-10-21

ORIGINAL RESEARCH

Time, Clocks, and the Ordering of Events in a Distributed System — Lamport, 1978 ↗

Ordering reference for Unit 14

Link checked · 2026-10-21

ORIGINAL RESEARCH

In Search of an Understandable Consensus Algorithm — Ongaro & Ousterhout ↗

Raft reference for Unit 15

Link checked · 2026-10-21

SUPPLIED STARTING REFERENCE

Transactional Outbox — supplied starting URL ↗

Verification pending before chapter production

Link verification pending

PATTERN AUTHOR'S EXPLANATION

Transactional Outbox — Chris Richardson ↗

Publication intent and duplicate delivery

Link checked · 2026-10-21

SUPPLIED STARTING REFERENCE

AWS Well-Architected: reliability pillar ↗

Verification pending before chapter production

Link verification pending

SUPPLIED STARTING REFERENCE

Google Cloud: disaster recovery planning ↗

Verification pending before chapter production

Link verification pending

ENGINEERING BOOKS

Google SRE books ↗

Reliability and production practice

Link checked · 2026-10-21

OFFICIAL DOCUMENTATION · ACCESSED 21 OCT 2026

OpenTelemetry concepts ↗

Telemetry reference for Unit 19

`Link checked · 2026-10-21`

SUPPLIED STARTING REFERENCE

OpenTelemetry: observability primer ↗

Verification pending before chapter production

`Link verification pending`

OFFICIAL DOCUMENTATION · ACCESSED 21 OCT 2026

Kubernetes components ↗

Platform reference for Unit 20

`Link checked · 2026-10-21`

ORIGINAL METHODOLOGY

The Twelve-Factor App ↗

Application deployment conventions

`Link checked · 2026-10-21`

SUPPLIED STARTING REFERENCE

AWS Well-Architected guidance ↗

Verification pending before chapter production

`Link verification pending`

SUPPLIED STARTING REFERENCE

AWS Well-Architected: cost optimization ↗

Verification pending before chapter production

`Link verification pending`

SUPPLIED STARTING REFERENCE

The Practical Test Pyramid — Ham Vocke ↗

Verification pending before chapter production

`Link verification pending`

ORIGINAL RESEARCH · ACM DOI

On the criteria to be used in decomposing systems into modules — D. L. Parnas, 1972 ↗

Module decomposition criteria; metadata and abstract verified

`Link checked · 2026-10-21`

No sources match that search.

AN HONEST COMPLETION MANIFEST

Mapped is different from produced

A visible record of scope prevents a syllabus, diagram plan, or source list from being mistaken for a finished textbook.

CURRENT INSTALLMENT

Blueprint complete; Unit 1 produced; Units 2–22 planned. The complete atlas is still in production.

Unit	Topics mapped	Chapters produced	Exercises / solutions
1 · Introduction to System Design	13	4 / 4	11 / 11
2 · Requirements, Estimation, and Engineering Trade-offs	15	0 / 4	0 / 0 · planned
3 · Networking Foundations for System Design	17	0 / 4	0 / 0 · planned
4 · Runtime, Processes, Threads, and Concurrency	17	0 / 4	0 / 0 · planned
5 · Object-Oriented and Modular Software Design	15	0 / 4	0 / 0 · planned
6 · Design Patterns, Dependency Injection, and Object Lifecycles	18	0 / 7	0 / 0 · planned
7 · Application Architecture and Domain-Driven Design	20	0 / 4	0 / 0 · planned
8 · API Design and Communication Contracts	19	0 / 4	0 / 0 · planned

Unit	Topics mapped	Chapters produced	Exercises / solutions
9 · Data Models and Storage Technologies	19	0 / 4	0 / 0 · planned
10 · Database Internals, Transactions, and Concurrency Control	18	0 / 4	0 / 0 · planned
11 · Caching and Content Delivery	17	0 / 4	0 / 0 · planned
12 · Scalability, Load Balancing, and Data Distribution	22	0 / 4	0 / 0 · planned
13 · File Systems, Search Infrastructure, and Data Pipelines	17	0 / 4	0 / 0 · planned
14 · Distributed Systems Fundamentals	20	0 / 4	0 / 0 · planned
15 · Replication, Leader Election, and Consensus	17	0 / 4	0 / 0 · planned
16 · Message Queues, Events, and Distributed Workflows	22	0 / 4	0 / 0 · planned
17 · Reliability, Fault Tolerance, and Disaster Recovery	22	0 / 4	0 / 0 · planned
18 · Security Architecture and Privacy	22	0 / 4	0 / 0 · planned
19 · Observability, Monitoring, and Incident Engineering	19	0 / 4	0 / 0 · planned
20 · Deployment, Cloud Infrastructure, and Platform Architecture	21	0 / 4	0 / 0 · planned
21 · Performance, Cost Optimization, and System Validation	20	0 / 4	0 / 0 · planned

Unit	Topics mapped	Chapters produced	Exercises / solutions
22 · End-to-End Design, Architecture Reviews, and Evolution	12 cases × 22 review sections	0 / 12	0 / 0 · planned

The production contract

1. Blueprint: allocate every topic, prerequisite, essential illustration, source, and case-study requirement.
2. Production: write a coherent chapter with worked execution, original visuals, failure behavior, and alternatives.
3. Review: check primary references, code, formulas, diagrams, cross-references, and assumptions.
4. Assembly: maintain indexes and a navigable reading edition; print only produced lessons and clearly identified reference material.

Four Unit 1 chapters, 16 figures, four interactive workbenches, and 11 questions with separate explanations are produced. Unit 1 implementation is downloadable. Units 2–22 and all mandatory case studies remain planned. This status is also embedded in the machine-readable manifest.